

LyDROP: WP-MEC 부분 오프로딩의 온라인 제어

Online Partial Computation Offloading Optimization in Wireless Powered Mobile Edge Computing Network

Lu Sun, Rina Liang, Liangtian Wan, Kaihui Liu, Zhaolong Ning, Jie Wang (2026)

무선 전력, 부분 에너지 분할, uplink 시간을 함께 정해 데이터 큐를 다룬다

왜 매 frame마다 다시 결정해야 하는가

channel과 새 task 도착이 변하면, 전력 수확과 데이터 처리의 균형도 함께 바뀐다

- | | | |
|----|-------------|--|
| 01 | 현재 channel | 수확 가능한 에너지 E 가 달라진다 |
| ↓ | | |
| 02 | 기다리는 data | 현재 queue Q 가 service의 가치를 바꾼다 |
| ↓ | | |
| 03 | 공유 frame | WPT 시간과 모든 uplink 시간이 경쟁한다 |
| ↓ | | |
| 04 | 온라인 control | 이번 frame의 δ , a , τ 를 선택한다 |

LyDROP은 고정된 offload 선택이 아니라 매 frame의 공동 자원 결정을 다룬다

한 frame의 물리적 분할: 전력 수확에서 service까지

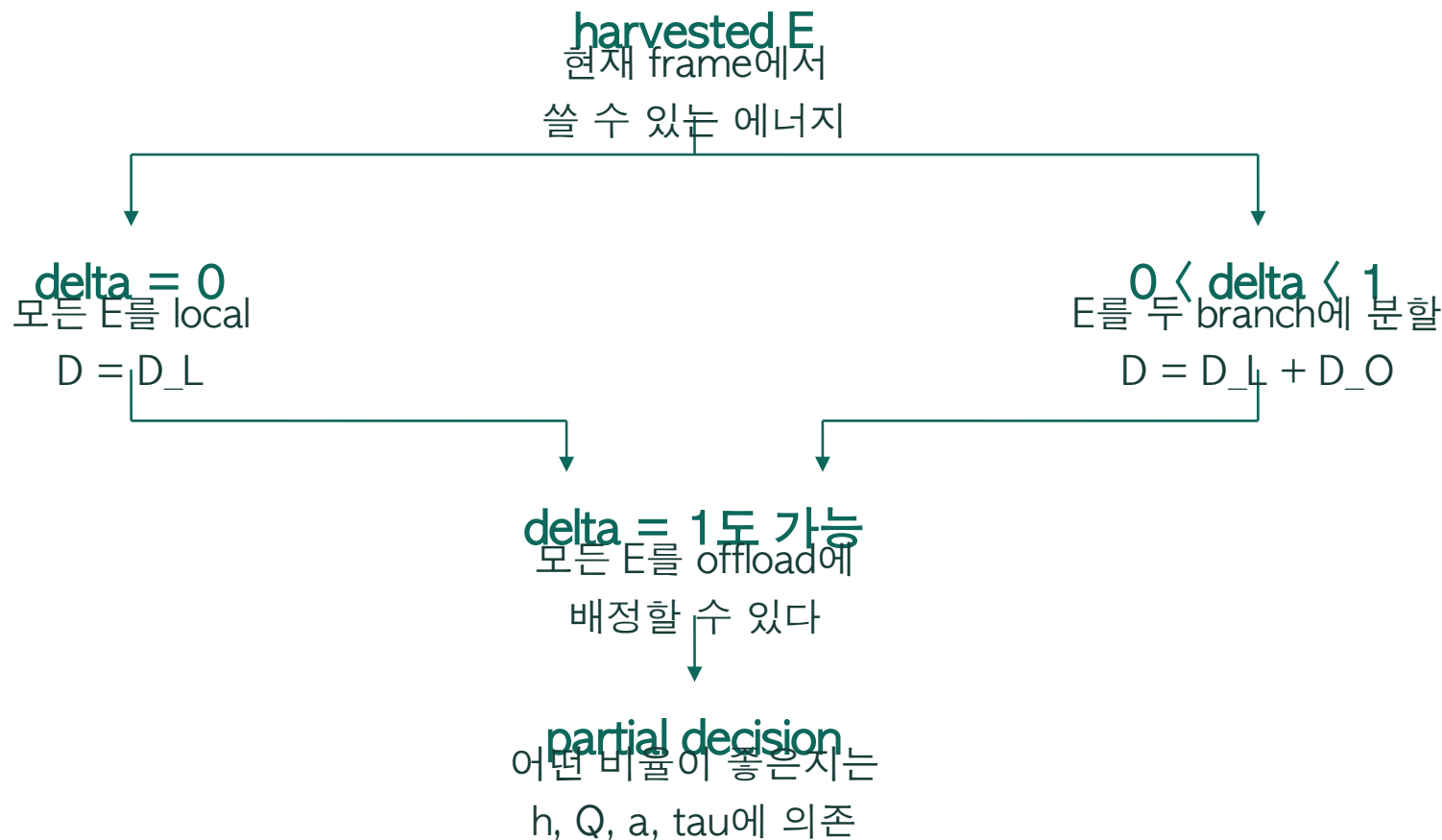
AP의 WPT 시간 a 가 에너지 E 를 만들고, E 와 uplink 시간 τ 가 local-offload service를 결정한다



a , δ , τ 는 서로 다른 자원 결정이며 하나의 offload label로 합치면 안 된다

delta: binary route가 아닌 연속 에너지 분할

0과 1 사이의 delta는 local과 offload 처리량을 같은 frame에 함께 만들 수 있다



service는 현재 data와 frame budget을 넘을 수 없다

새 arrival은 이번 service 뒤 queue에 들어오므로 actor의 현재 state와 구분된다

제약 또는 상태	의미	잘못 읽기 쉬운 점
$a + \sum \tau \leq 1$	WPT와 모든 WD uplink가 한 frame을 공유	각 WD가 τ 를 독립 선택하지 않는다
$D_i \leq Q_i(t)$	현재 queue에 있던 data만 service 가능	이번 arrival을 미리 처리하지 않는다
$Q_i(\text{next}) = Q_i - D_i + R_i$	service 뒤 새 arrival을 더한 data queue	virtual energy queue가 아니다
actor state x_i	current h 와 Q	R 은 actor input으로 명시되지 않는다

queue update의 순서를 지켜야 partial offloading의 인과관계가 보인다

강의용 한 WD 계산: 분할·제약·queue를 함께 확인한다

아래 수치는 논문 실험값이 아니라 식의 인과관계를 보이는 재계산 예시다

01	energy	$a=0.5$ 이면 $E=0.5 \times 4 \times 1 \times 0.5 \times 1 = 1$ J
↓		
02	split	$\delta=0.5$ 로 local과 offload에 각 0.5 J
↓		
03	service	$D_L=1.587$, $D_O=1.098$, total $D=2.685$ bits
↓		
04	causal checks	$a+\tau=0.75 \leq 1$, $D=2.685 \leq Q=3$
↓		
05	next queue	arrival $R=0.2$ 뒤 $Q(\text{next})=3-2.685+0.2=0.515$

수확 에너지, shared time, 현재 queue 검사를 통과한 뒤에만 새 arrival을 더한다

P1의 장기 목표를 P2의 현재 frame score로 읽는다

Lyapunov drift-plus-penalty는 queue backlog와 weighted computation rate를 같은 frame 결정에 넣는다

수준	원문의 질문	발표에서 읽을 핵심
P1 long run	weighted rate를 높이고 strong data-queue stability를 만족	장기 목표와 제약을 정의한다
Lyapunov L	data queue 제곱합으로 congestion을 표현	큰 Q는 미처리 data가 많음을 뜻한다
P2 per frame	$Q_i + V \omega_i$ 로 현재 rate를 가중	backlog와 rate 선호를 함께 반영
$V > 0$	penalty scaling parameter	실험 tuning 관찰을 보장으로 바꾸지 않는다

P2는 이번 frame의 candidate를 비교하는 score이며 learned-policy 최적성 정리는 아니다

Q와 V omega가 P2에서 하는 일

P2는 한 WD의 score를 Q_i 와 V omega_i가 곱해진 current computation rate로 높이려 한다

- Q_i 가 크면 오래 기다린 data를 service하는 항의 비중이 커진다.
- omega_i는 device별 weighted-rate priority를 표현한다.
- V는 rate objective의 scale을 조절하는 control parameter다.
- 이 score만으로 모든 future channel-arrival을 예측하거나 learned policy의 안정성을 증명하지는 않는다.

P2는 backlog를 반영한 current-frame rate 기준이다

LyDROP: DNN 제안과 model-based 선택

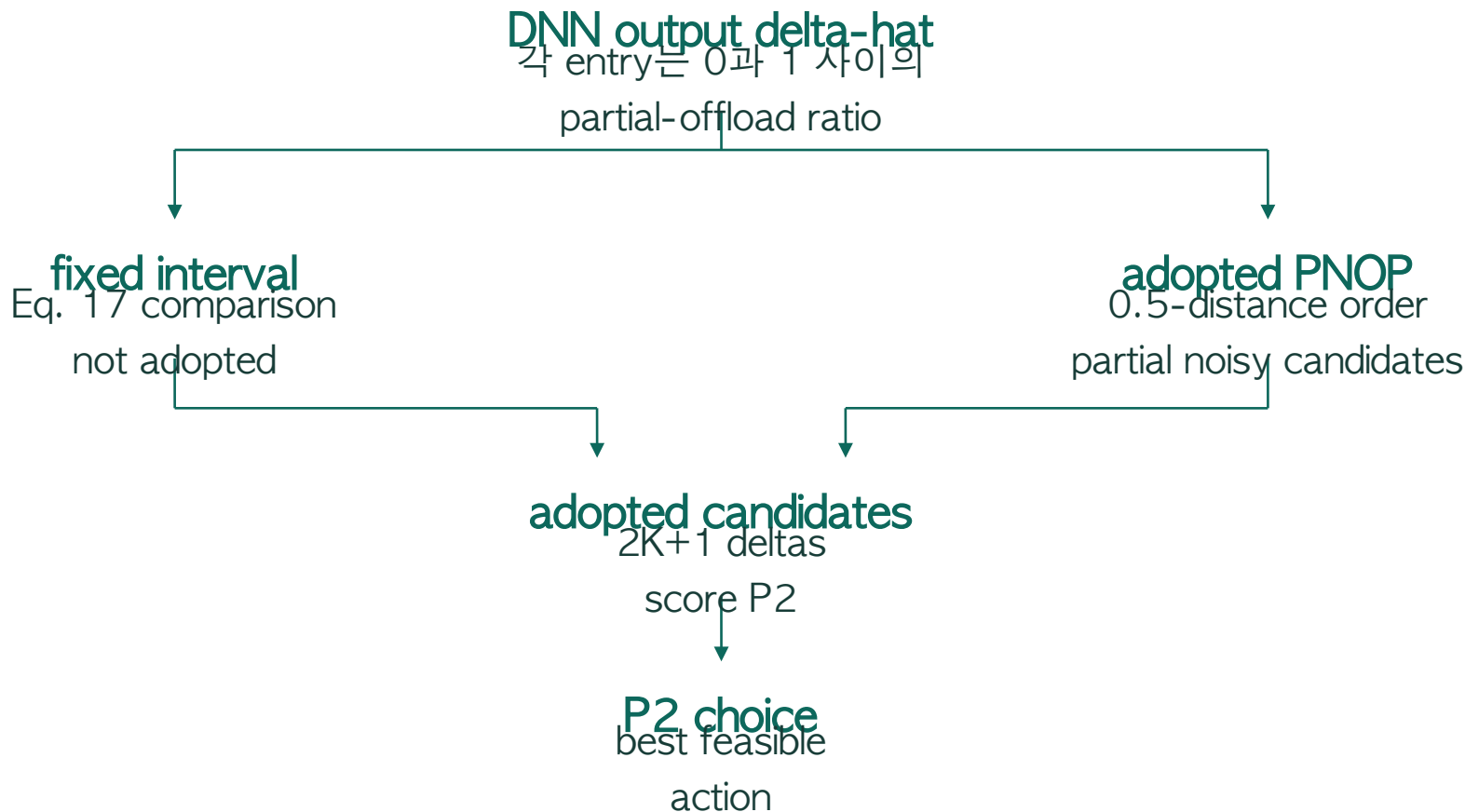
actor는 relaxed delta를 제안하고, critic은 candidate별 P2 resource solve로 최종 joint action을 고른다



critic은 learned value network가 아니라 fixed-delta resource optimizer다

two constructions: fixed vs adopted PNOP

원문은 두 construction을 설명하지만, actual control은 adopted partial NOP candidate set을 쓴다



noisy order: near-0.5 entry의 순서를 사용

교육용 vector는 후보가 여전히 연속 partial ratio임을 보여 주며, 원문의 수치 예시는 아니다

01	relaxed vector	$\hat{\Delta} = \{0.18, 0.47, 0.82\}$
↓		
02	distance	0.5와의 거리로 entry order를 만든다
↓		
03	shifted branch	ranked entry에 partial shift를 적용한다
↓		
04	noise branch	Gaussian white noise를 더해 candidate를 넓힌다
↓		
05	ratio candidates	각 entry가 local-offload energy split을 유지

candidate search는 binary action 조합 나열이 아니라 partial vector 주변의 탐색이다

fixed delta 뒤의 연속 resource solve

candidate delta가 고정되면 a와 tau를 푸는 P2 subproblem은 convexity를 이용할 수 있다



fixed-delta solver는 candidate 평가용 P2 subproblem을 푼다

bisection으로 dual epsilon을 찾는다

Algorithm 2의 목적은 candidate별 timing resource allocation을 빠르게 계산하는 것이다

01 **bounds** LB와 UB로 epsilon search interval을 둔다



02 **midpoint** epsilon을 midpoint로 잡고 condition Omega를 계산



03 **bound update** condition 부호에 따라 LB 또는 UB를 갱신



04 **stop** $|UB-LB|$ 가 tolerance 이하면 멈춘다



05 **resources** a^* 와 τ^* 를 계산해 G evaluation에 쓴다

Lambert-W 유도식의 상세 암기는 필요 없지만 fixed-delta 이후의 계산 역할은 알아야 한다

online action과 periodic DNN training

Algorithm 1은 매 frame의 joint action과 replay-memory batch training을 같은 순서에 두지만 역할은 구분한다

과정	언제 쓰는가	무엇을 하는가
online action	각 frame의 current h, Q	delta candidate를 평가해 δ^* , τ^* , a^* 실행
queue update	현재 service 뒤	D를 반영하고 새 R을 더해 Q 갱신
replay memory	joint action 실행 뒤	Fig. 3의 (x_i, δ) 경험으로 memory 갱신
DNN training	$\text{mod}(t, \gamma)=0$ 일 때	갱신된 memory의 sample batch로 학습

training interval γ 와 on-device training latency는 다른 문제다

실험 결과는 simulator 설정 안에서 읽어야 한다

paper의 rate-queue plots는 WP-MEC field deployment measurement가 아니라 명시된 simulation protocol의 비교다

항목	원문 조건	해석 경계
implementation	TensorFlow 2.13, Python 3.9	framework version 자체가 deployment proof는 아니다
system	$B=2$ MHz, $P_a=3$ W, $T=1$ s, $V=36$	Table III의 parameter setting이다
channel and arrivals	i.i.d. Rician h , equal-mean exponential arrival	실제 traffic trace 일반화가 아니다
locations	AP와 WD positions fixed in simulation	mobility와 multi-AP를 직접 검증하지 않는다

그래프의 우위는 source가 지정한 channel, arrival, topology 조건에 묶여 있다

Table II: noisy choice의 rate-queue 결과

논문은 noisy order-preserving method의 weighted rate가 더 높고 queue가 acceptable하다고 해석한다

quantization	weighted sum rate	average data queue length
order-preserving	7.6573	2.0842
noisy order-preserving	7.9523	3.4037

noisy method는 이 table에서 higher rate를 보였지만 lower queue를 보인 것은 아니다

K와 V는 paper의 simulation tuning 변수다

candidate count와 drift-penalty scale은 이 연구의 설정에서 실험적으로 선택된다

- Table IV의 $K=8, 9, 10$ CPU computation time은 각각 0.1833, 0.1636, 0.1551이다.
- Fig. 5와 Table IV를 바탕으로 authors는 $K=10$ 을 선택한다.
- Fig. 6은 $\text{kappa}_i=8$ Mbps에서 V 가 커질수록 weighted rate가 증가하는 실험 관찰을 보인다.
- $K=10$ 과 특정 V 가 모든 topology·arrival·hardware에서 최적이라는 뜻은 아니다.

parameter selection은 source 조건의 empirical choice이며 일반 규칙이나 theorem이 아니다

baseline은 queue를 어떤 방식으로 반영하는지에서 다르다

각 baseline 설명은 이 source의 experiment definition에 한정해 읽는다

방법	partial decision 출발점	queue와 resource 처리
LyDROP	DNN proposal과 candidate quantization	P2 score로 candidate를 평가
LyCD	random partial decisions	coordinate descent와 dual timing solve
LyRPO	random partial strategies	Lyapunov P2 formulation을 사용
Myopic	current weighted rate	queue backlog를 objective에서 무시

baseline은 candidate 생성과 queue objective를 따로 비교한다

N 비교: trace와 CPU time을 구분

Figs. 7–9의 rate-queue traces와 Table V의 CPU computation time은 같은 종류의 지표가 아니다

evidence	원문 조건 또는 값	발표에서의 올바른 읽기
Figs. 7–9	$N=10, 20, 30$; $n=10000$	각 simulator trace의 rate와 queue 비교
Fig. 7 setting	$\kappa_i=8$ Mbps, $\omega_i=1$	priority를 넣지 않은 stated condition
Table V	LyCD/LyDROP ratio: 3.68 at $N=20$, 16.39 at $N=30$	program execution time ratio
interpretation	N 이 증가한 experiments	MEC end-to-end latency claim이 아니다

CPU program time은 wireless service latency와 다른 지표다

priority와 arrival sweep도 조건을 붙여 읽는다

source는 weight가 다른 WD와 arrival rate가 다른 scenario에서 rate·queue를 비교한다

experiment	원문 조건	해석 경계
priority	even WD $\omega=1.2$, odd WD $\omega=1$	특정 weight assignment의 simulation
arrival sweep	mean arrival을 바꾸어 Figs. 12–13 비교	offered load 변화 실험
Myopic queue	8에서 8.25 Mbps로 갈 때 at least two orders larger	unstable points are omitted later
reported result	LyDROP rate·queue plots	모든 stochastic traffic의 guarantee가 아니다

priority weight와 arrival rate를 말하지 않으면 plot의 queue 차이를 일반화하기 쉽다

Melbourne: location-based simulation

real user locations를 활용했지만 WPT field deployment measurement로 읽을 근거는 없다

- source는 Melbourne BS 위치를 정하고 주변 latitude-longitude 범위의 816 user groups를 사용한다.
- Figs. 14–15는 user count를 바꾸며 네 알고리즘을 비교한다.
- location distribution이 균등 거리 가정과 다르다는 점을 experiment input에 반영한다.
- 실제 AP charging, device battery, MEC execution trace를 현장에서 측정한 결과라고 주장하지 않는다.

real location data와 field deployment evidence는 서로 다른 수준의 근거다

모델 가정은 결과를 어디까지 일반화할지 정한다

paper의 positive result와 system omission을 함께 읽어야 LyDROP의 적용 경계를 알 수 있다

가정 또는 omission	왜 중요한가
same-frame energy use	controller state에 inter-frame battery dynamics를 두지 않는다
one AP, fixed WD locations	mobile·multi-AP coordination을 직접 다루지 않는다
AP compute and feedback delay omitted	task complexity·concurrency가 커지면 schedule 판단이 달라질 수 있다
Lyapunov-guided DNN candidate control	learned policy의 global optimality theorem을 source가 제시하지 않는다

LyDROP은 명시된 WP-MEC model에서의 방법이며 모든 deployment 조건의 보장은 아니다

LyDROP을 이해하는 네 연결

부분 에너지 분할, data queue, candidate selection, fixed-delta solver가 함께 있어야 논문의 control loop가 완성된다

- | | | |
|----|---------------------|---|
| 01 | physical split | a가 E를 만들고 delta가 E를 local-offload에 나눈다 |
| ↓ | | |
| 02 | queue guidance | Q와 V omega가 P2에서 현재 service 가치를 만든다 |
| ↓ | | |
| 03 | candidate selection | DNN delta-hat에서 $2K+1$ partial vector를 생성한다 |
| ↓ | | |
| 04 | resource solve | fixed delta마다 dual/bisection으로 a^* , τ^* , G를 계산한다 |

LyDROP은 partial split, queue control, P2 solve를 결합한다